gemstone-rs Shared Core Integration

How gemstone-py-native can wrap the Rust core

Generated from the Markdown sources in gemstone-rs/docs.

Contents

Shared Core Integration

The long-term direction is:

```
gemstone-gci
  Low-level dynamic libgcirpc loader and raw GCI calls.

gemstone-rs
  Safe Rust API: Config, Session, Oop, Value, browser, codegen, BridgeRoot.
  Also exposes gemstone_rs::py_native as a narrow adapter contract.

gemstone-py-native
  Thin PyO3 wrapper around the Rust core.

gemstone-py
  Python API, async/web adapters, examples, docs, and compatibility layer.
```

This keeps Python and Rust from growing separate native bridges.

What Should Move Into Rust

- dynamic GCI library loading
- OOP constants and conversions
- session login/logout
- eval, execute, perform
- global get/put
- string, symbol, array, and dictionary marshalling
- transaction commit/abort
- browser primitives
- codegen discovery primitives

What Should Stay Python-Specific

- Pythonic `Session` and async wrappers
- FastAPI, Litestar, Django examples
- Python package extras such as `gemstone-py[fast]`
- backward-compatible return behavior
- Python docs and notebooks

PyO3 Wrapper Shape

`gemstone-py-native` should become a small adapter:

```
Python call -> PyO3 wrapper -> gemstone-rs Session -> gemstone-gci -> libgcirpc
```

The Rust side now exposes `gemstone_rs::py_native`, which is deliberately plain Rust and dependency-free. A PyO3 crate can wrap these types without depending on internal `Session` details:

- `PyNativeConfig`
- `PyNativeConfigSummary`
- `PyNativeValue`
- `PyNativeErrorInfo`
- `PyNativeSession`
- `capabilities()`

Print the adapter contract from the CLI when a wrapper build, CI job, or documentation generator needs a stable machine-readable surface:

```
gemstone-rs py-native capabilities
gemstone-rs py-native capabilities --json
```

The JSON output is covered by `schemas/gemstone-rs.py-native.schema.json` and the smoke output is covered by `schemas/gemstone-rs.py-native-smoke.schema.json`. Value/error samples are covered by `schemas/gemstone-rs.py-native-samples.schema.json`. The migration checklist output is covered by `schemas/gemstone-rs.py-native-migration.schema.json`. The Python compatibility shim output is covered by `schemas/gemstone-rs.py-native-compat.schema.json`. The wrapper conformance output is covered by `schemas/gemstone-rs.py-native-conformance.schema.json`. The downstream handoff bundle output is covered by `schemas/gemstone-rs.py-native-handoff.schema.json`. The publish receipt output is covered by `schemas/gemstone-rs.py-native-publish-receipt.schema.json`. The one-command downstream gate output is covered by `schemas/gemstone-rs.py-native-check-all.schema.json`. The VS Code workbench packages these schemas for editor validation. A checked-in fixture lives at `examples/py-native/gemstone-rs.py-native.json`, the value/error sample fixture lives at `examples/py-native/gemstone-rs.py-native-samples.json`, and the dry-run smoke fixture lives at `examples/py-native/gemstone-rs.py-native-smoke.json`, with the compatibility fixture at `examples/py-native/gemstone-rs.py-native-compat.json`, and the conformance fixture at `examples/py-native/gemstone-rs.py-native-conformance.json`, with the final handoff bundle at `examples/py-native/gemstone-rs.py-native-handoff.json`, and the native wheel publish receipt at `examples/py-native/gemstone-rs.py-native-publish-receipt.json`, so downstream wrapper CI can diff the contract without requiring a live stone. Use the CLI check in CI:

```
gemstone-rs py-native check examples/py-native/gemstone-rs.py-native.json
gemstone-rs py-native check examples/py-native/gemstone-rs.py-native.json --json
gemstone-rs py-native check-samples examples/py-native/gemstone-rs.py-native-
samples.json
gemstone-rs py-native check-samples examples/py-native/gemstone-rs.py-native-
samples.json --json
gemstone-rs py-native check-smoke examples/py-native/gemstone-rs.py-native-smoke.json
gemstone-rs py-native check-smoke examples/py-native/gemstone-rs.py-native-smoke.json
--json
gemstone-rs py-native smoke --dry-run
gemstone-rs py-native smoke --dry-run --json
gemstone-rs py-native migration
gemstone-rs py-native migration --json
gemstone-rs py-native compatibility
gemstone-rs py-native compatibility --json
gemstone-rs py-native check-compat examples/py-native/gemstone-rs.py-native-
compat.json
gemstone-rs py-native conformance
gemstone-rs py-native conformance --json
gemstone-rs py-native check-conformance examples/py-native/gemstone-rs.py-native-
conformance.json
gemstone-rs py-native handoff
gemstone-rs py-native handoff --json
gemstone-rs py-native check-handoff examples/py-native/gemstone-rs.py-native-
handoff.json
gemstone-rs py-native publish-receipt
gemstone-rs py-native publish-receipt --json
gemstone-rs py-native check-publish-receipt examples/py-native/gemstone-rs.py-native-
publish-receipt.json
gemstone-rs py-native check-all
gemstone-rs py-native check-all --json
```

`py-native migration --json` is intentionally not a replacement for doing the Python-side work. It is the shared Rust-side checklist that CLI, CI, docs, and the VS Code workbench can render consistently while `gemstone-py-native` keeps its additive `RustCoreSession` bridge over `gemstone_rs::py_native` green through live smoke and published-wheel verification. `py-native handoff --json` is the single manifest to hand to downstream `gemstone-py-native` work: it lists each artifact, schema, generation command, validation command, and required acceptance check. `py-native publish-receipt --json` records the verified TestPyPI and PyPI workflow runs, install commands, and package checks for the Rust-backed `gemstone-py-native` wheel release. `py-native check-all --json` is the CI acceptance gate for that handoff: it validates the checked-in capabilities, samples, smoke, compatibility, conformance, handoff, and publish-receipt fixtures in one report.

To create a starter PyO3 wrapper crate from the installed CLI:

```
gemstone-rs examples scaffold py_native_pyo3_adapter ./gemstone-py-native-starter
cd ./gemstone-py-native-starter
cargo run
python -m venv .venv
```

```
source .venv/bin/activate
python -m pip install maturin pytest
maturin develop
python -c 'import gemstone_py_native; print(gemstone_py_native.migration_json())'
python -c 'import gemstone_py_native; print(gemstone_py_native.compatibility_json())'
python -c 'import gemstone_py_native; print(gemstone_py_native.conformance_json())'
python -c 'import gemstone_py_native; print(gemstone_py_native.handoff_json())'
python -c 'import gemstone_py_native_compat;
print(gemstone_py_native_compat.compatibility_report()["returnPolicy"])'
pytest
```

The scaffold writes `pyproject.toml`, `src/lib.rs`, `PYTHON.md`, `python/gemstone_py_native_compat.py`, and a Python smoke test. It is deliberately thin: Python calls PyO3 functions/classes, and those delegate into `gemstone_rs::py_native`. The generated crate currently uses PyO3 0.28 for Python 3.14 compatibility and keeps PyO3's `extension-module` flag behind a Cargo feature so `cargo run` works while `maturin develop` still builds a proper Python extension. It exposes `capabilities_json`, `samples_json`, `smoke_dry_run_json`, `migration_json`, `compatibility_json`, `conformance_json`, and `handoff_json`, so Python wrapper CI can inspect the adapter contract, compatibility method map, conformance target, handoff manifest, and live/publish verification checklist from the generated module. The generated `NativeSession` also maps the Rust session operations for eval, execute, resolve, value-to-OOP conversion, perform, strings, symbols, globals, export-set retention, and transactions without adding Pythonic return conversion in the native layer. It includes JSON value helpers such as `eval_json` and `perform_json` so Python code can translate the stable `PyNativeValue` shape without reimplementing Rust value classification. The generated compatibility shim demonstrates the package-layer policy: direct object identity becomes `OopHandle`, raw native OOPs stay below the package boundary, typed value-to-OOP helpers are explicit opt-in methods, and value-returning calls become dictionaries decoded from the Rust JSON contract. The CLI exposes the same method map as `py-native compatibility --json`, including every generated Python method, the underlying `NativeSession` method, the native return type, and the Python return type. The generated `conformance_json()` output is the higher-level integration target: extension module functions, raw `NativeSession` methods, compatibility shim methods, fixture paths, and scaffold files.

From a gemstone-rs source checkout, verify that the embedded scaffold still compiles against the local Rust core with:

```
python3 scripts/check_py_native_pyo3_scaffold.py
```

Run the dry-run contract check from a source checkout when you also want to exercise the example binary:

```
cargo run -p gemstone-rs --example python_native_adapter -- --dry-run
```

Run it against a live stone:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
cargo run -p gemstone-rs --example python_native_adapter
```

The wrapper should expose stable operations first:

- `login(config)`
- `eval(source)`
- `execute(source)`
- `value_to_oop(value)`
- `perform(oop, selector, args)`
- `commit()`
- `abort()`
- `logout()`

Only after that should it expose higher-level browser, codegen, and BridgeRoot operations.

Downstream PyO3 Shape

The downstream `gemstone-py-native` crate should stay thin:

```
use gemstone_rs::py_native::{PyNativeConfig, PyNativeSession};

struct NativeSession {
    inner: PyNativeSession,
}

impl NativeSession {
    fn login(config: PyNativeConfig) -> gemstone_rs::Result<Self> {
        Ok(Self {
            inner: PyNativeSession::login(config)?,
        })
    }

    fn eval(&mut self, source: &str) -> gemstone_rs::Result<String> {
        Ok(format!("{:?}", self.inner.eval(source)?))
    }
}
```

Actual PyO3 code should translate `PyNativeValue` into Python objects and translate `PyNativeErrorInfo` into Python exceptions. Keep `PyNativeSession` unsendable unless a dedicated worker-thread wrapper is used.

Migration Plan

1. Keep `gemstone-rs` independent and publishable.
2. Scaffold or adapt the starter PyO3 crate with `gemstone-rs examples

   scaffold py_native_pyo3_adapter`.

3. Keep `scripts/check_py_native_pyo3_scaffold.py` green while the starter

   evolves.

4. Keep the existing `gemstone-py-native` `RustCoreSession` bridge delegating

   to `gemstone_rs::py_native`.

5. Continue reducing duplicated native loading code in `gemstone-py-native`.
6. Run the existing `gemstone-py` native backend and live tests through the

   Rust-backed native path.

7. Keep pure Python fallback behavior and current sync return behavior

   backward compatible.

The main design rule: Rust owns the native bridge; Python owns Python ergonomics.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.